



FLACSO
ECUADOR

SESIÓN 5: INTRODUCCIÓN A SQL

SQL: consultas sobre bases de datos relacionales

Fausto Jácome Pérez
FLACSO Ecuador

Ya sabemos transformar datos con Polars. ¿Cómo hacemos las mismas operaciones directamente sobre una base de datos?

¿Dónde estamos?

3-4

SESIONES ANTERIORES

ETL con Polars: ingesta, limpieza, joins, reshaping, agrupaciones y window functions sobre archivos locales.

5

ESTA SESIÓN

SQL: las mismas operaciones en otro lenguaje, directamente sobre una base de datos PostgreSQL en la nube. Solo lectura.

6

PRÓXIMA SESIÓN

SQL: modificar datos con INSERT, UPDATE y DELETE. Crear y alterar tablas con DDL. Cada quien trabaja en su propio schema.

Las tablas son las mismas del registro escolar ficticio. La diferencia es que ahora viven en un servidor PostgreSQL en Supabase, no en un archivo local.

¿Qué es una base de datos relacional?

Archivo plano (CSV, Excel)

- Un archivo = una tabla
- Sin restricciones sobre los datos
- Sin control de acceso por tabla
- Se carga completo en memoria para operar
- No hay forma de consultar sin leerlo todo

Base de datos relacional

- Múltiples tablas con relaciones definidas
- Tipos, llaves y restricciones de integridad
- Control de acceso por usuario y objeto
- El motor ejecuta las consultas sin cargar todo en memoria
- Se puede consultar desde cualquier cliente o lenguaje

Una base de datos relacional es un sistema que almacena datos en tablas estructuradas, define relaciones entre ellas y garantiza su integridad. SQL es el lenguaje para interactuar con ella.

Structured Query Language (SQL): un lenguaje, muchos motores

SQL es un estándar ISO. La sintaxis central es la misma en todos los motores; las diferencias son detalles de funciones específicas o extensiones propietarias.

Motores open source

PostgreSQL

MySQL / MariaDB

SQLite

DuckDB

Motores comerciales y cloud

SQL Server

Oracle

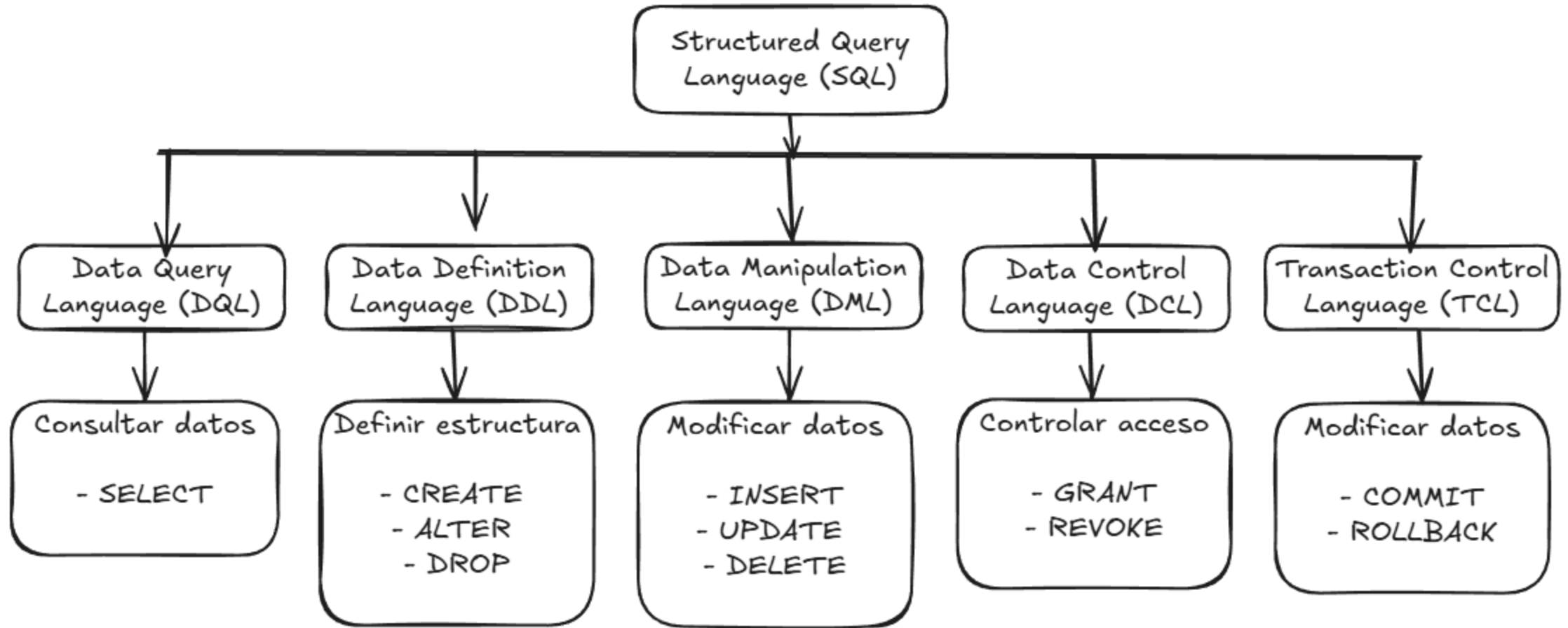
BigQuery

Redshift

Snowflake

Lo que aprendemos hoy en PostgreSQL es directamente transferible a cualquier otro motor. Las diferencias más comunes son en funciones de fecha, manejo de texto y algunas extensiones de window functions.

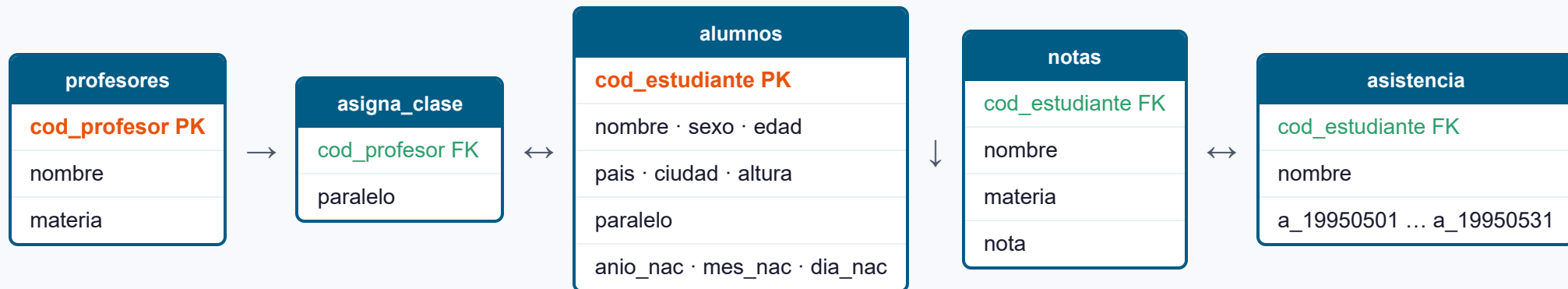
Las familias de comandos SQL



Schemas, tablas y relaciones

Una base de datos contiene uno o más esquemas (**schema**) con nombre, que a su vez contienen tablas. Los esquemas también contienen otros tipos de objetos con nombre, como tipos de datos, funciones y operadores.

En esta clase el schema `public` contiene todas las tablas compartidas.



naranja = llave primaria (PK) · verde = llave foránea (FK) ·

Tipos de datos en PostgreSQL

Tipo PostgreSQL	Descripción	Equivalente Polars
INTEGER	Entero de 32 bits	pl.Int32
BIGINT	Entero de 64 bits	pl.Int64
NUMERIC	Decimal de precisión exacta	pl.Decimal
REAL / FLOAT	Decimal de punto flotante	pl.Float32 / Float64
TEXT	Cadena de texto sin límite	pl.String
VARCHAR(n)	Cadena con límite de longitud	pl.String
BOOLEAN	TRUE / FALSE	pl.Boolean
DATE	Fecha sin hora	pl.Date
TIMESTAMP	Fecha con hora	pl.Datetime

En PostgreSQL `NUMERIC` sin parámetros acepta cualquier valor decimal sin pérdida de precisión. `REAL` es más rápido pero puede tener errores de redondeo.

Consultas básicas

SELECT, FROM, WHERE, ORDER BY, LIMIT

Estructura de una consulta SQL

El orden de escritura y el orden de ejecución son distintos. SQL evalúa las cláusulas de adentro hacia afuera, no de arriba hacia abajo:

```
SELECT  nombre, edad, ciudad      -- 5. selecciona columnas del resultado
FROM    alumnos                  -- 1. identifica la tabla
WHERE   pais = 'Ecuador'         -- 2. filtra filas
GROUP BY ciudad                  -- 3. agrupa
HAVING  COUNT(*) > 1             -- 4. filtra grupos
ORDER BY edad DESC               -- 6. ordena el resultado
LIMIT  10;                      -- 7. limita filas devueltas
```

El orden de escritura es fijo: `SELECT → FROM → WHERE → GROUP BY → HAVING → ORDER BY → LIMIT .`

SELECT y FROM

```
-- Todas las columnas
SELECT * FROM alumnos;

-- Columnas específicas
SELECT nombre, edad, ciudad FROM alumnos;

-- Alias con AS: renombrar columnas en el resultado
SELECT nombre AS alumno, edad AS edad_anios, altura/100 AS altura_m FROM alumnos;

-- Columna de otra tabla calificando con el nombre de tabla
SELECT alumnos.nombre, notas.materia, notas.nota
FROM notas
JOIN alumnos ON notas.cod_estudiante = alumnos.cod_estudiante;
```

`SELECT *` es útil para explorar, pero en producción es mejor listar columnas explícitamente. Columnas con mayúsculas requieren comillas dobles: `SELECT "NOMBRE_APELLIDOS" FROM nuevos_alumnos;`

WHERE

-- Comparación simple

```
SELECT * FROM alumnos WHERE pais = 'Ecuador';
```

-- Operadores: = <> < > <= >=

```
SELECT * FROM notas WHERE nota >= 7.0;
```

-- AND y OR

```
SELECT * FROM alumnos  
WHERE pais = 'Ecuador' AND edad > 25;
```

```
SELECT * FROM alumnos  
WHERE pais = 'Ecuador' OR pais = 'Colombia';
```

-- Equivalente más limpio con IN

```
SELECT * FROM alumnos  
WHERE pais IN ('Ecuador', 'Colombia', 'Peru');
```

-- Valores nulos

```
SELECT * FROM notas WHERE nota IS NULL;  
SELECT * FROM notas WHERE nota IS NOT NULL;
```

WHERE con rangos y negación

```
-- BETWEEN: equivale a >= y <=, ambos extremos incluidos
SELECT * FROM notas WHERE nota BETWEEN 6.0 AND 8.0;

-- NOT IN: excluir un conjunto de valores
SELECT * FROM alumnos WHERE pais NOT IN ('Ecuador', 'Colombia');

-- NOT: negar cualquier condición
SELECT * FROM alumnos WHERE NOT (edad > 30);

-- Combinar condiciones con paréntesis para controlar precedencia
SELECT * FROM alumnos WHERE (pais = 'Ecuador' OR pais = 'Colombia') AND edad > 25;
```

AND tiene mayor precedencia que **OR**. Sin paréntesis, **A OR B AND C** se evalúa como **A OR (B AND C)**. Usar paréntesis siempre que haya mezcla de operadores lógicos.

Texto: LIKE, ILIKE y regex

```
-- LIKE: sensible a mayúsculas. % = cualquier secuencia, _ = un carácter
SELECT * FROM alumnos WHERE nombre LIKE 'Ana%';
SELECT * FROM alumnos WHERE nombre LIKE '%ez'; -- termina en ez
SELECT * FROM alumnos WHERE nombre LIKE '_uiz'; -- Ruiz, Luiz, etc.

-- ILIKE: igual pero insensible a mayúsculas (extensión PostgreSQL)
SELECT * FROM alumnos WHERE nombre ILIKE 'ana%';

-- Regex con ~* (insensible a mayúsculas)
-- Equivalente a str.contains() con (?i) en Polars
SELECT * FROM viaje
WHERE cuotas_pagadas ~* '^(s[íi]|afirmativo|cumplido|positivo)$';

-- Versión compacta: empieza con s, a, c o p
SELECT * FROM viaje
WHERE cuotas_pagadas ~* '^[sacp]';
```

~* usa expresiones regulares POSIX, la misma lógica que `str.contains()` en Polars. ~ es sensible a mayúsculas; ~* no.

ORDER BY y LIMIT

-- Ascendente (por defecto)

```
SELECT * FROM alumnos ORDER BY edad;
```

-- Descendente

```
SELECT * FROM alumnos ORDER BY edad DESC;
```

-- Múltiples columnas: primero por paralelo, luego por nombre

```
SELECT * FROM alumnos ORDER BY paralelo, nombre;
```

-- Nulls: por defecto van al final en DESC, al inicio en ASC

-- Para controlar explícitamente:

```
SELECT * FROM notas ORDER BY nota DESC NULLS LAST;
```

-- LIMIT: devolver solo las primeras N filas

```
SELECT * FROM alumnos ORDER BY edad DESC LIMIT 5;
```

-- OFFSET: saltar las primeras N filas (paginación)

```
SELECT * FROM alumnos ORDER BY cod_estudiante LIMIT 10 OFFSET 10;
```

Transformaciones

Expresiones calculadas, CAST y CASE WHEN

Expresiones calculadas y CAST

```
-- Aritmética directa en SELECT
SELECT nombre,
       altura / 100.0                AS altura_m,
       ROUND(altura / 100.0, 2)      AS altura_m_r,
       (altura - AVG(altura) OVER ()) /
       NULLIF(STDDEV(altura) OVER (), 0) AS altura_z
FROM alumnos;

-- CAST: cambiar tipo de dato
SELECT CAST(cod_estudiante AS TEXT)      AS id_texto,
       CAST('2026-01-01' AS DATE)       AS fecha,
       CAST(nota AS INTEGER)            AS nota_entera
FROM notas;

-- Sintaxis alternativa de cast con :: (exclusiva de PostgreSQL)
SELECT cod_estudiante::TEXT, nota::INTEGER
FROM notas;
```

`NULLIF(expr, 0)` devuelve NULL si el valor es 0, evitando división por cero. Útil al calcular z-scores o tasas.

CASE WHEN

Equivalente directo al `pl.when().then().otherwise()` de Polars:

```
-- Recodificación simple
SELECT nombre,
       nota,
       CASE WHEN nota >= 7.0 THEN 'aprobado'
            WHEN nota >= 5.0 THEN 'recuperacion'
            ELSE 'reprobado'
       END AS estado
FROM notas;

-- Equivalente a la recodificación de cuotas_pagadas con regex
SELECT cod_estudiante,
       cuotas_pagadas,
       CASE WHEN cuotas_pagadas ~* '^[sacp]' THEN TRUE
            ELSE FALSE
       END AS cuotas_ok
FROM notas;
```

Las condiciones del `CASE` se evalúan en orden. La primera que se cumpla gana. Si ninguna coincide y no hay `ELSE`, el resultado es `NULL`.

Agrupaciones

GROUP BY, funciones de agregación y HAVING

GROUP BY y funciones de agregación

```
-- Edad media por sexo (equivalente al group_by().agg() de Polars)
```

```
SELECT sexo,  
       AVG(edad)    AS edad_media,  
       COUNT(*)     AS n_alumnos  
FROM alumnos  
GROUP BY sexo  
ORDER BY sexo;
```

```
-- Nota media y conteo por paralelo
```

```
SELECT paralelo,  
       ROUND(AVG(nota), 2) AS nota_media,  
       COUNT(nota)        AS n_notas      -- cuenta solo no nulos  
FROM notas  
JOIN alumnos USING (cod_estudiante)  
GROUP BY paralelo  
ORDER BY paralelo;
```

`COUNT(*)` cuenta todas las filas del grupo incluyendo nulls. `COUNT(columna)` cuenta solo las filas donde esa columna no es nula. La diferencia equivale a `p1.len()` vs `p1.col().count()`.

Funciones de agregación

Función	Descripción	Equivalente Polars
COUNT(*)	Filas totales del grupo	p1.len()
COUNT(col)	Valores no nulos	p1.col().count()
SUM(col)	Suma	p1.col().sum()
AVG(col)	Media aritmética	p1.col().mean()
MIN(col)	Mínimo	p1.col().min()
MAX(col)	Máximo	p1.col().max()
STDDEV(col)	Desviación estándar	p1.col().std()
STRING_AGG(col, sep)	Concatenar valores	p1.col().str.join(sep)

HAVING

HAVING filtra **grupos** después de la agregación. **WHERE** filtra **filas** antes de agrupar.

```
-- Solo paralelos con más de 3 notas registradas
SELECT paralelo,
       ROUND(AVG(nota), 2) AS nota_media,
       COUNT(nota)        AS n_notas
FROM notas
JOIN alumnos USING (cod_estudiante)
GROUP BY paralelo
HAVING COUNT(nota) > 3
ORDER BY nota_media DESC;

-- Combinando WHERE y HAVING
SELECT materia,
       ROUND(AVG(nota), 2) AS promedio
FROM notas
WHERE nota IS NOT NULL      -- filtra filas antes de agrupar
GROUP BY materia
HAVING AVG(nota) > 5.0      -- filtra grupos después de agrupar
ORDER BY promedio DESC;
```

Joins

Combinar tablas por claves comunes

Tipos de join

Tipo	¿Qué conserva?	Cuándo usarlo
INNER JOIN	Solo filas con coincidencia en ambas tablas	Subconjunto garantizado
LEFT JOIN	Todas las filas de la tabla izquierda	Agregar columnas sin perder filas base
FULL JOIN	Todas las filas de ambas tablas	Diagnóstico: ver qué falta en cada lado
IN / EXISTS	Filas izquierda con coincidencia, sin columnas nuevas	Filtrar por existencia (semi join)
NOT IN / NOT EXISTS	Filas izquierda sin coincidencia	Encontrar registros sin pareja (anti join)

Antes de cualquier join: verificar que la llave no tiene duplicados en la tabla derecha con `SELECT cod_estudiante, COUNT(*) FROM notas GROUP BY cod_estudiante HAVING COUNT(*) > 1` . Un duplicado multiplica silenciosamente las filas.

INNER JOIN y LEFT JOIN

-- INNER JOIN: solo alumnos con notas registradas

```
SELECT a.nombre, n.materia, n.nota
FROM notas n
INNER JOIN alumnos a ON n.cod_estudiante = a.cod_estudiante;
```

-- LEFT JOIN: todos los alumnos, con o sin notas

```
SELECT a.nombre, n.materia, n.nota
FROM alumnos a
LEFT JOIN notas n ON a.cod_estudiante = n.cod_estudiante;
```

-- USING: atajo cuando la columna llave tiene el mismo nombre en ambas tablas

```
SELECT a.nombre, n.materia, n.nota
FROM alumnos a
LEFT JOIN notas n USING (cod_estudiante);
```

-- JOIN con múltiples columnas llave

```
SELECT *
FROM notas n
LEFT JOIN profesores p ON n.materia = p.materia;
```


FULL JOIN

Conserva todas las filas de ambas tablas. Las filas sin pareja quedan con `NULL` en las columnas del otro lado:

```
-- Diagnóstico: ¿qué alumnos originales ya estaban en los nuevos?
SELECT
  a.cod_estudiante,
  a.nombre,
  b."COD_ESTUDIANTE"      AS cod_nuevo,
  CASE WHEN b."COD_ESTUDIANTE" IS NOT NULL
        THEN TRUE ELSE FALSE
  END                     AS nuevo_estudiante
FROM alumnos a
FULL JOIN nuevos_alumnos b
  ON a.cod_estudiante = b."COD_ESTUDIANTE";
```

Si `b."COD_ESTUDIANTE"` es `NULL` después del full join, la fila solo existe en la tabla izquierda. Si `a.cod_estudiante` es `NULL`, solo existe en la tabla derecha.

Semi join con IN y EXISTS

Filtran filas de la tabla izquierda **sin agregar columnas**. Equivalen al `how="semi"` de Polars:

```
-- Con IN: alumnos que tienen al menos una nota registrada
SELECT *
FROM alumnos
WHERE cod_estudiante IN (
    SELECT cod_estudiante FROM notas
);

-- Con EXISTS: equivalente, más eficiente en tablas grandes
SELECT *
FROM alumnos a
WHERE EXISTS (
    SELECT 1 FROM notas n
    WHERE n.cod_estudiante = a.cod_estudiante
);
```

`EXISTS` se detiene en cuanto encuentra la primera coincidencia. `IN` materializa toda la sublista primero. Para tablas pequeñas la diferencia es irrelevante; para tablas grandes `EXISTS` suele ser más rápido.

Anti join con NOT IN y NOT EXISTS

Conservan las filas que **no tienen** coincidencia. Equivalen al `how="anti"` de Polars:

```
-- Con NOT IN: alumnos que NO tienen notas registradas
SELECT *
FROM alumnos
WHERE cod_estudiante NOT IN (
    SELECT cod_estudiante FROM notas
    WHERE cod_estudiante IS NOT NULL -- precaución con nulls
);

-- Con NOT EXISTS: equivalente, sin el problema de nulls
SELECT *
FROM alumnos a
WHERE NOT EXISTS (
    SELECT 1 FROM notas n
    WHERE n.cod_estudiante = a.cod_estudiante
);
```

`NOT IN` devuelve vacío si la subquery contiene algún `NULL`. Usar siempre `WHERE columna IS NOT NULL` dentro de la subquery, o preferir `NOT EXISTS` que no tiene ese problema.

Window functions

OVER, PARTITION BY y funciones de ventana

Concepto: OVER y PARTITION BY

`OVER()` calcula un agregado por partición y lo asigna a cada fila sin reducirlas. Es el equivalente directo de `.over()` en Polars y de `PARTITION BY` en todos los motores SQL:

```
-- Media de nota por materia asignada a cada fila
SELECT nombre,
       materia,
       nota,
       AVG(nota) OVER (PARTITION BY materia) AS media_materia,
       STDDEV(nota) OVER (PARTITION BY materia) AS std_materia
FROM notas;
```

nombre	materia	nota	media_materia
Raul Martos	castellano	0.0	2.81
Santos Rico	castellano	2.27	2.81
Raul Martos	matematicas	3.73	6.87

Z-score con window functions

Exactamente el mismo cálculo que hicimos con `.over()` en Polars:

```
SELECT nombre,  
       materia,  
       nota,  
       ROUND(  
         (nota - AVG(nota) OVER (PARTITION BY materia)) /  
         NULLIF(STDDEV(nota) OVER (PARTITION BY materia), 0), 2) AS z_score  
FROM notas  
ORDER BY materia, z_score DESC;
```

Polars

```
pl.col("nota").mean().over("materia")  
pl.col("nota").std().over("materia")
```

SQL

```
AVG(nota) OVER (PARTITION BY materia)  
STDDEV(nota) OVER (PARTITION BY materia)
```

Agregar con OVER sin PARTITION BY

Sin `PARTITION BY`, la ventana es toda la tabla. Útil para calcular proporciones sobre el total:

```
-- Porcentaje que representa cada nota sobre el total de notas
SELECT nombre,
       materia,
       nota,
       SUM(nota) OVER () AS total_notas,
       ROUND(nota / NULLIF(SUM(nota) OVER (), 0) * 100, 2) AS pct
FROM notas
WHERE nota IS NOT NULL;

-- Combinar partición global y por grupo en la misma query
SELECT nombre,
       materia,
       nota,
       AVG(nota) OVER () AS media_global,
       AVG(nota) OVER (PARTITION BY materia) AS media_materia
FROM notas;
```

ROW_NUMBER, RANK y DENSE_RANK

Funciones de numeración de filas dentro de cada partición:

```
SELECT nombre,
       materia,
       nota,
       ROW_NUMBER() OVER (PARTITION BY materia ORDER BY nota DESC) AS fila,
       RANK() OVER (PARTITION BY materia ORDER BY nota DESC) AS ranking,
       DENSE_RANK() OVER (PARTITION BY materia ORDER BY nota DESC) AS ranking_denso
FROM notas
ORDER BY materia, nota DESC;
```

Función	Empates	Saltos
ROW_NUMBER()	Asigna números consecutivos sin importar empates	No aplica
RANK()	Mismo número en empate	Salta posiciones después del empate
DENSE_RANK()	Mismo número en empate	No salta posiciones

¿Qué aprendimos hoy?

Conceptos

- Base de datos relacional vs archivo plano
- SQL como estándar independiente del motor
- Las cinco familias: DDL, DQL, DML, DCL, TCL
- Schema, tabla, llave primaria y foránea
- Orden de escritura vs orden de ejecución

En SQL / PostgreSQL

- `SELECT / FROM / WHERE / ORDER BY / LIMIT`
- `CAST` y operador `::`
- `CASE WHEN` para recodificación
- `GROUP BY / HAVING` con `COUNT`, `AVG`, `SUM`
- `INNER JOIN`, `LEFT JOIN`, `FULL JOIN`
- Semi join con `IN` / `EXISTS`
- Anti join con `NOT IN` / `NOT EXISTS`
- `OVER (PARTITION BY)` para window functions
- `ROW_NUMBER`, `RANK`, `DENSE_RANK`